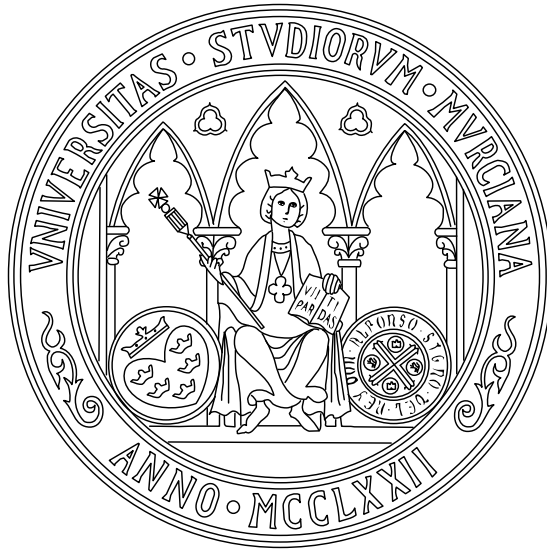




UNIVERSIDAD DE MURCIA
Facultad de Informática

PRÁCTICAS DE
Informática Gráfica

4º DE GRADO EN INGENIERÍA INFORMÁTICA

PROFESOR: GINÉS GARCÍA MATEOS

CURSO 2025/2026

Hugo Sánchez Martínez

24450997L

`hugo.s.m@um.es`

Contenidos

1. Introducción	1
2. Requisitos	1
3. Tecnologías	1
4. Características generales	2
4.1. Menú Archivo	2
4.2. Menú Edición	2
4.3. Menú Herramientas	3
4.4. Menú Transformar	3
4.5. Menú Efectos	4
5. Desarrollo de las implementaciones	5
5.1. Nueva imagen desde el portapapeles	5
5.2. Copiar al portapapeles	6
5.3. Deshacer última acción	7
5.4. Información multimedia	9
5.5. Ecualización del histograma	11
5.6. Modelos de color	13
5.7. Perfilado	15
5.8. Ajuste de los canales rojo, verde y azul	17
6. Mejoras adicionales no mencionadas en el enunciado	19
6.1. Incremento del paso en arcoíris	19
6.2. Rehacer última acción	21
7. Bitácora	23
A. Anexo	24
A.1. Código de la implementación de Información Multimedia	24
A.2. Código de la implementación de convertir_a_modelo()	26
A.3. Código de la implementación de perfilado()	27
A.4. Código de la implementación de ver_rojoverdeazul()	28

Índice de figuras

1.	Actualización de la estructura ventana con un vector	img_hist	7
2.	Diálogo de información multimedia implementado en QFotoPaint6.		10
3.	Actualización de la estructura ventana con un vector	undo_hist	21

Resumen

Este documento especifica la implementación de las prácticas de la asignatura de Informática Gráfica del último curso del grado en Ingeniería Informática de la Universidad de Murcia.

Dicha práctica consiste en el desarrollo de una aplicación de procesamiento multimedia, basada en herramientas y funcionalidades presentes en aplicaciones de edición de imágenes y vídeo ya existentes. El objetivo de este documento es describir en detalle el proceso de desarrollo de la aplicación, explicando las funcionalidades implementadas durante la práctica.

1. Introducción

QFotoPaint es una aplicación de escritorio para el procesamiento de imágenes y vídeo desarrollada en C++, utilizando el *framework* Qt y la librería OpenCV.

Inspirada en herramientas reales de edición como *GIMP* o *Adobe Photoshop*, QFotoPaint permite al usuario abrir, visualizar y editar múltiples imágenes simultáneamente, aplicando diferentes filtros, transformaciones y efectos de procesamiento digital de imagen de forma sencilla e intuitiva.

2. Requisitos

QFotoPaint requiere tener instalado un compilador compatible con el estándar C++17 o superior.

Es necesario disponer del entorno de desarrollo Qt Creator, junto con el framework Qt y la librería OpenCV, correctamente configurados para compilar y ejecutar la aplicación.

3. Tecnologías

Durante el desarrollo de QFotoPaint se han utilizado las siguientes tecnologías:

- **Git:** Para el control de versiones y la gestión colaborativa del código fuente.
- **Qt Creator:** Entorno de desarrollo integrado (IDE) empleado para la codificación, depuración y diseño de la interfaz gráfica.
- **Qt Framework:** Biblioteca multiplataforma usada para la creación de la interfaz de usuario y la gestión de eventos.
- **OpenCV:** Librería de visión por computador utilizada para el procesamiento, filtrado y transformación de imágenes y vídeo.
- **qmake:** Herramienta de automatización utilizada para la configuración y compilación del proyecto Qt.

4. Características generales

El programa base proporcionado por los profesores de la asignatura cuenta con un gran número de herramientas, de las cuales algunas han sido modificadas durante el transcurso de las prácticas. Esta sección del documento tiene como finalidad mostrar el uso y correcto funcionamiento de las principales funcionalidades de la aplicación.

4.1. Menú Archivo

Nueva

Crea una nueva imagen, especificando el tamaño y el color de fondo.

Abrir

Lee una imagen de un archivo existente en formatos admitidos por OpenCV.

Guardar

Guarda la imagen actual sin requerir una nueva especificación de nombre.

Guardar como...

Guarda la imagen actual, permitiendo especificar un nuevo nombre de archivo.

Cerrar

Cierra la imagen abierta, dando opción a guardar si ha sido modificada.

Capturar de cámara

Captura una imagen de una cámara de vídeo conectada al ordenador.

Capturar de vídeo

Abre un frame específico de un archivo de vídeo y permite previsualizar la posición.

Salir

Termina el programa, liberando memoria y dando opción a guardar las imágenes modificadas.

4.2. Menú Edición

Seleccionar todo

Selecciona la imagen completa como región de interés (ROI).

Copiar a nueva

Crea una nueva imagen conteniendo solo la región de interés (ROI) seleccionada.

Ver histograma

Muestra el histograma de la imagen, ya sea en valor de gris o por canales (R, G, B).

Opciones

Configura ajustes del programa como el color actual y la opción de preguntar si guardar.

4.3. Menú Herramientas**Punto**

Pinta puntos en la imagen con tamaño, color y nivel de difuminación dados.

Línea

Pinta líneas en la imagen especificando un punto de inicio y otro de fin.

Rectángulo

Pinta rectángulos (borde o relleno) en la imagen usando las esquinas especificadas.

Elipses

Pinta elipses (borde o relleno) seleccionando los extremos o el centro y el tamaño.

Arco iris

Dibuja puntos cuyo color cambia automáticamente y rotativamente siguiendo una paleta de tipo arco iris.

Seleccionar

Define una región de interés (ROI) rectangular para usar en operaciones posteriores.

4.4. Menú Transformar**Ajustar brillo/contraste/gama**

Modifica la imagen alterando el brillo (suma), el contraste (multiplicación) y la gama.

Ajustar matiz/saturación/luminosidad

Realiza un ajuste de color usando HLS/HSV para modificar el matiz, la saturación y la luminosidad.

Ajuste lineal del histograma

Estira linealmente el histograma indicando el porcentaje de píxeles a descartar en los extremos.

Perspectiva

Realiza una transformación perspectiva mapeando 4 puntos origen a 4 puntos destino seleccionados por el usuario.

Convertir a escala de color

Convierte la imagen a escala de grises o a color según la opción seleccionada.

Media ponderada de dos imágenes

Calcula una nueva imagen como la media ponderada de dos imágenes, escalando si los tamaños no coinciden.

4.5. Menú Efectos**Suavizado**

Aplica un filtro de suavizado (gaussiano, media o mediana) según el tipo y tamaño indicados.

Bajorrelieve

Aplica un efecto de bajorrelieve a la imagen, seleccionando la dirección y cantidad del efecto.

Pinchar/estirar

Aplica transformaciones geométricas de pinchar o estirar una imagen indicando centro, radio y cantidad del efecto.

Inpaint

Aplica el algoritmo inpaint para borrar contenido de la imagen pintando sobre las zonas afectadas.

5. Desarrollo de las implementaciones

El siguiente apartado documenta el trabajo de ampliación realizado sobre el código base proporcionado. En contraste con las herramientas preexistentes, esta sección detalla las funcionalidades adicionales ofertadas por el enunciado de la práctica, explicando el diseño implementado, los cambios técnicos realizados en la estructura del programa y la validación de su correcto funcionamiento.

5.1. Nueva imagen desde el portapapeles

Esta es una de las operaciones adicionales más sencillas. No es necesario manejar el API del portapapeles de Windows a bajo nivel, pues el framework de Qt ofrece una clase “*wrapper*” que permite acceder al portapapeles de forma sencilla y ofrece los métodos necesarios para la funcionalidad que se desea implementar. [1]

Por simplicidad, se ha optado por no crear una nueva clase ni un nuevo formulario, simplemente definimos una nueva `QAction` en el menú “Archivo” y accedemos al portapapeles de la siguiente forma:

```
1 // mainwindow.cpp
2 void MainWindow::on_actionNueva_desde_portapapeles_triggered() {
3     const QClipboard *clipboard = QGuiApplication::clipboard();
4     const QMimeData *mimeTypeData = clipboard->mimeTypeData();
5
6     if (mimeTypeData->hasImage()) {
7         QImage img = qvariant_cast<QImage>(mimeTypeData->imageData());
8
9         if (!img.isNull()) {
10             Mat mat = Mat(img.height(), img.width(), CV_8UC4, img.bits());
11             cvtColor(mat, mat, COLOR_RGBA2RGB);
12
13             int pl = comprobar_primera_libre();
14             if (pl != -1)
15                 crear_nueva(pl, mat);
16         }
17     } else {
18         QMessageBox::warning(this, "Portapapeles vacío",
19                             "No se encontró ninguna imagen en el portapapeles.");
20     }
21 }
```

Listing 1: Acceso al portapapeles en Qt.

La aplicación lanzará un diálogo de advertencia si el portapapeles está vacío o no se ha encontrado una imagen en la última entrada del historial.

5.2. Copiar al portapapeles

Siguiendo la dinámica de la anterior característica relacionada con el portapapeles, esta funcionalidad permite al usuario copiar la imagen de la ventana seleccionada al portapapeles. Esta operación no necesita la creación de una nueva clase ni de un nuevo formulario, y se puede implementar en el propio código de `mainwindow.cpp`.

```
1 // mainwindow.cpp
2 void MainWindow::on_actionCopiar_al_portapapeles_triggered() {
3     int f_act = foto_activa();
4
5     if (f_act == -1)
6         return;
7
8     ventana &v = foto[f_act];
9
10    Mat roi = v.img(v.roi).clone();
11
12    Mat roiRGB;
13    cvtColor(roi, roiRGB, COLOR_BGR2RGB);
14
15    QImage img = QImage((const uchar*)roiRGB.data, roiRGB.cols, roiRGB.rows,
16    roiRGB.step, QImage::Format_RGB888).copy();
17
18    QClipboard *clipboard = QApplication::clipboard();
19    clipboard->setImage(img);
20 }
```

Listing 2: Acceso al portapapeles en Qt.

Realizamos una conversión del modo BGR de OpenCV al modo de color RGB. Se ha intentado convertir al modo de color RGBA pero la imagen contenía errores a la hora de recuperarla. Para actualizar el portapapeles, se hace uso de la operación `setImage()` proporcionada por la clase `QClipboard` de Qt. [2]

5.3. Deshacer última acción

5.3.1. Introducción

Los humanos cometemos numerosos errores y, como cualquier programa de usuario, QFotoPaint6 debería permitir al usuario deshacer los últimos cambios. Esta funcionalidad ha sido una de las más difíciles de implementar, ya que está diseñada de forma que se pueda revertir cualquier cambio cometido en la imagen, ya sea con un formulario o con una herramienta de edición.

Lo primero que nos viene a la mente es definir, para cada ventana (imagen) abierta, una lista con el historial de acciones realizadas, para poder navegar hacia una de ellas. Sin embargo, en vez de guardar la acción realizada, se guarda el “estado” de la imagen. Para llevarlo a cabo, será necesario modificar la estructura `ventana`, añadiendo un tipo de dato adecuado para soportar las operaciones.

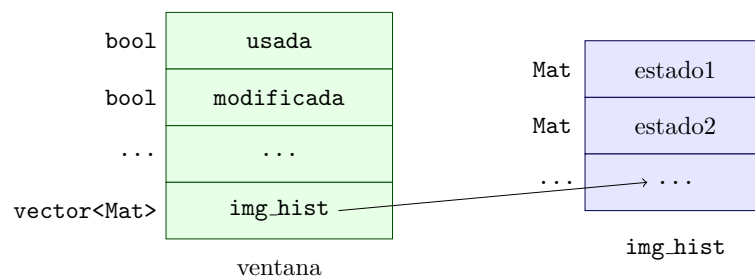


Figura 1: Actualización de la estructura `ventana` con un vector `img_hist`.

5.3.2. Implementación

Para implementarlo en código, modificamos en `imagenes.h`, añadiendo el nuevo vector dinámico que actuará como pila, junto con una constante que limitará el tamaño del historial:

```

1 // imagenes.h
2 #define MAX_HIST_IMG 20 // Tamaño del historial para deshacer
3 ...
4 struct ventana {
5     bool usada;           // Indica si se está usando actualmente
6     bool modificada;      // Ha sido modificada después de abrirla o crearla
7     string nombre;        // Nombre de la ventana codificada en UTF8
8     string nombref;       // Nombre del fichero de imagen codificado en Latin1
9     int orden;            // Número de orden entre las ventanas abiertas
10    Rect roi;              // Región de interés seleccionada
11    Mat img;               // Imagen almacenada en esta posición
12    vector<Mat> img_hist;  // Historial de imágenes
13 };

```

Listing 3: Modificación del `struct ventana`.

Es importante definir métodos que permitan implementar las acciones que deseamos soportar. Para ello, se definen los métodos `guardar_estado()`:

```

1 // imagenes.cpp
2 void guardar_estado (int nfoto) {
3     ventana& v = foto[nfoto];
4
5     v.img_hist.push_back(v.img.clone()); // guardar estado en la pila de
6     deshacer
7
8     if (v.img_hist.size() > MAX_HIST_IMG) // comprobar tamaño máximo de pila
9         v.img_hist.erase(v.img_hist.begin());
10
11     v.modificada = true;
12 }
```

y `deshacer()`:

```

1 // imagenes.cpp
2 void deshacer (int nfoto) {
3     ventana& v = foto[nfoto];
4     ...
5     // recuperar y eliminar último estado guardado
6     v.img = v.img_hist.back().clone();
7     v.img_hist.pop_back();
8
9     imshow(v.nombre, v.img);
10 }
```

Para cada acción del menú, el paso a seguir es el mismo para una: guardar el estado de la imagen en cada cambio de la ventana. Para ello, añadimos una llamada a `guardar_estado()` en cada condicional `if (guardar)`. Un ejemplo es el siguiente:

```

1 void ver_brillo_contraste_gamma (int nfoto, double suma, double prod, double
2     gamma, bool guardar) {
3     ...
4     if (guardar) {
5         guardar_estado(nfoto); // instrucción añadida
6         img.copyTo(foto[nfoto].img);
7         foto[nfoto].modificada= true;
8     }
9 }
```

Sin embargo, si queremos que también funcione para las herramientas (punto, línea, arcoíris), al funcionar con eventos, hay que controlar el inicio y fin de “dibujado”. Esto se logra con una variable `dibujando_activo` que se actualiza en el *callback*, según los eventos `EVENT_LBUTTONDOWN` y `EVENT_LBUTTONUP`.

```

1 void callback (int event, int x, int y, int flags, void *_nfoto) {
2     ...
3     // 1.5. Se inicia la pulsación del ratón
4     if (event==EVENT_LBUTTONDOWN) {
5         downx= x;
6         downy= y;
7
8         if (!dibujando_activo) { // marcar inicio trazo
9             guardar_estado(factual);
10            dibujando_activo = true;
11        }
12    }
13
14    if (event==EVENT_LBUTTONUP) { // finalizar trazo
15        if (dibujando_activo)
16            dibujando_activo = false;
17    }
18 }

```

De esta forma, cuando el usuario usa, por ejemplo, la herramienta “Punto” y mantiene presionado el botón del ratón, solo se guardará un estado de la imagen: el previo a realizar la acción. Además, se ha asignado al QAction correspondiente el atajo de teclado `CTRL+Z`, aunque no funciona cuando el foco está en una de las ventanas de OpenCV.

5.4. Información multimedia

5.4.1. Introducción

Otra funcionalidad adicional es la de poder consultar información acerca de las imágenes abiertas. Datos como las dimensiones, el tipo o la relación de aspecto pueden ser útiles para el usuario. Para ello, es necesario crear una nueva clase, junto con un formulario, `InformacionMultimedia`.



Observación Se ha optado por representar todas las imágenes abiertas en lugar de únicamente la imagen seleccionada, debido a la simplicidad de la implementación, ya que basta con recorrer el arreglo de imágenes `foto[]` de `struct ventana`.

El formulario diseñado ha sido inspirado en el utilizado por el programa VLC para consultar estadísticas e información acerca de los *streams* de los ficheros multimedia.

5.4.2. Implementación

El diálogo implementado tiene el siguiente aspecto:

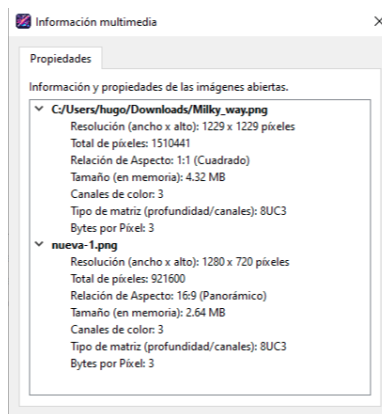


Figura 2: Diálogo de información multimedia implementado en QFotoPaint6.

El diseño se ha logrado usando un `QTabWidget` junto con un `QTreeWidget`. Para extender el contenido de éste al superior, se ha usado un *layout* vertical. Esta implementación ofrece una gran comodidad a la hora añadir elementos a su nodo raíz en la clase *src*. A nivel de código, esto se ha logrado de la siguiente forma:

1. Se recorren todas las imágenes abiertas:

```
for (size_t i = 0; i < MAX_VENTANAS; ++i) {
    if (foto[i].img.empty()) continue; ... }
```

2. Se crea un nodo raíz:

```
QTreeWidgetItem *root = new QTreeWidgetItem(ui->treeWidget);
```

3. Se van creando los nodos hijo, cada uno con una propiedad sobre la imagen:

```
QTreeWidgetItem *itemCanales = new QTreeWidgetItem(root);
itemCanales->setText(0, QString("Canales de color: %1").arg(img.channels() ));
```

Para ver el código en su totalidad, se remite al anexo de la implementación de la clase [A.1](#).

5.5. Ecualización del histograma

5.5.1. Introducción

La ecualización de histogramas es un método utilizado para mejorar el contraste de una imagen, al expandir las zonas que concentran mucho contraste. Su objetivo principal es transformar una imagen de entrada en otra imagen cuyo histograma tiene una distribución uniforme, aunque luego veremos que esto no es exactamente así, pues los histogramas no son completamente planos. [2]

La ecualización de histogramas no funciona igual en imágenes con un solo canal (imágenes de grises) que en imágenes a color. Para entender su funcionamiento, consideramos una imagen de grises X , siendo n_i el número de ocurrencias para el nivel de grises i . La probabilidad por tanto de que un píxel aleatorio tenga el valor de i es:

$$p_X(i) = \frac{n_i}{n}, \quad 0 \leq i < L$$

siendo L el número total de niveles de grises en la imagen. $p_X(i)$ es el valor del histograma para i .

Definimos también la distribución acumulada de píxeles en X para el valor i .

$$\text{cdf}_X(i) = \sum_{j=0}^i p_X(j)$$

Para que esta función pueda emplearse como transformación de intensidades, debe normalizarse de manera que su valor máximo corresponda al nivel máximo de intensidad permitido (OpenCV usa 255). Es aquí donde empleamos el método que nos proporciona la librería de OpenCV `cv::equalizeHist`. [3]

equalizeHist()

void cv::equalizeHist (InputArray src, OutputArray dst)

Ecualiza el histograma de una imagen en escala de grises.

La función ecualiza el histograma de la imagen de entrada utilizando el siguiente algoritmo:

- Calcular el histograma H para `src`.
- Normalizar el histograma de manera que la suma de los bins sea 255.
- Calcular el histograma acumulado:

$$H'_i = \sum_{0 \leq j < i} H(j)$$

- Transformar la imagen utilizando H' como tabla de consulta:

$$\text{dst}(x, y) = H'(\text{src}(x, y))$$

El algoritmo normaliza el brillo y aumenta el contraste de la imagen.

5.5.2. Implementación

El desarrollo de la ecualización del histograma está fuertemente basada en la del ajuste lineal realizado en clase. Según la documentación, la funcionalidad debe permitir al usuario dos modos:

- **Conjunto:** La operación se aplica de manera conjunta sobre un grupo completo de canales. Concretamente, se realizará sobre un conjunto específico (valor en HSV y luminancia en YCrCb). Es necesario convertir a uno de estos modos ya que, como se especifica en la teoría, pueden aparecer resultados erróneos si usamos el canal RGB, el cual mezcla el color con el brillo. [4]
- **Independiente:** Si la imagen de entrada consta de varios canales, cada uno se procesa por separado sin tener en cuenta a los demás.

Para implementar correctamente la funcionalidad, se ha optado por crear una nueva clase junto con un formulario, `EcualizacionHistograma`. El método implementado en `imagenes.cpp` es el siguiente:

```

1  // imagenes.cpp
2  void ecualizacion_hist(int nfoto, int modo, bool guardar) {
3  Mat img = foto[nfoto].img;
4  Mat imres;
5
6  if (img.channels() == 1) { // imagen de grises
7      equalizeHist(img, imres);
8  } else if (modo == 0) { // modo conjunto
9      // convertir a HSV para separar luz del color
10     Mat hsv;
11     cvtColor(img, hsv, COLOR_BGR2HSV);
12
13     vector<Mat> canales;
14     split(hsv, canales);
15
16     equalizeHist(canales[2], canales[2]);
17
18     merge(canales, hsv);
19     cvtColor(hsv, imres, COLOR_HSV2BGR);
20 } else { // modo independiente
21     vector<Mat> canales;
22     split(img, canales);
23
24     for (size_t i = 0; i < 3; ++i)
25         equalizeHist(canales[i], canales[i]);
26
27     merge(canales, imres);
28 }
29
30 ...
31
32 }
```

Listing 4: Implementación de la ecualización del histograma

Como se puede ver en las líneas 10-16 del código 4, se realiza una conversión de modelo a HSV para separar la luminosidad de la crominancia.

5.6. Modelos de color

5.6.1. Introducción

Los modelos de color son representaciones matemáticas que describen cómo se codifican, combinan y reproducen los colores en distintos sistemas físicos o digitales, asignando valores numéricos a componentes fundamentales como RGB en pantallas o CMYK en impresión.

A continuación se describen brevemente los modelos que se van a implementar para esta funcionalidad:

- **RGB:** *Red, Green, Blue* — Rojo, Verde y Azul.
- **HSV:** *Hue, Saturation, Value* — Matiz, Saturación y Valor.
- **HLS:** *Hue, Lightness, Saturation* — Matiz, Luminosidad y Saturación.
- **XYZ:** Espacio definido por la CIE, donde X, Y y Z son componentes tricromáticos normalizados.
- **YUV:** *Y* (Luma), *U* y *V* (componentes de cromaticidad).
- **YCrCb:** Variante derivada de YUV; *Y* (Luma), *Cr* (Cromaticidad roja), *Cb* (Cromaticidad azul).
- **CMYK:** *Cyan, Magenta, Yellow, Key (Black)* — Cian, Magenta, Amarillo y Negro.

5.6.2. Conversión a CMYK

A diferencia del resto de modelos mencionados, OpenCV no ofrece una operación para convertir al modelo de color CMYK de forma directa (no existe un `COLOR_BGR2CMYK`). Para ello, tendremos que realizar la conversión de forma manual.

La conversión de un modelo RGB (en realidad BGR, ya que usamos OpenCV) a CMYK implica primero transformar los valores de intensidad luminosa $R, G, B \in [0, 1]$ a sus complementos sustractivos C', M', Y' . El vector de entrada en RGB se denota como: [5]

$$t_{\text{RGB}} = \{R, G, B\}$$

La conversión inicial a CMY aplica complementos directos:

$$t_{\text{CMY}} = \{C', M', Y'\} = \{1 - R, 1 - G, 1 - B\}$$

En el modelo CMYK se introduce un cuarto componente K (negro). El objetivo típico es maximizar el uso de tinta negra y minimizar la de CMY, lo cual mejora la estabilidad del color y reduce el consumo total de tinta. Para obtener CMYK se evalúa primero el mínimo entre los componentes CMY intermedios:

$$K = \min\{C', M', Y'\}$$

Si el mínimo es igual a 1, se define:

$$t_{\text{CMYK}} = \{0, 0, 0, 1\}$$

En caso contrario, cada componente de color se normaliza restando K y escalando por el rango restante $1 - K$. Así, la fórmula general queda:

$$t_{\text{CMYK}} = \left\{ \frac{C' - K}{1 - K}, \frac{M' - K}{1 - K}, \frac{Y' - K}{1 - K}, K \right\}$$

5.6.3. Implementación

Esta funcionalidad se ha implementado mediante QActions en el menú, en lugar de crear un formulario, debido a su sencillez. En primer lugar, definimos un nuevo `enum` en `imagenes.h`, que declarará los modelos de color soportados para convertir, junto con el prototipo de la función que contendrá la lógica de conversión:

```

1  enum modelo_color {
2      MC_RGB,      // Rojo, Verde, Azul
3      MC_HSV,      // Matiz, Saturación, Valor
4      MC_HLS,      // Matiz, Luminosidad, Saturación
5      MC_XYZ,      // CIE XYZ
6      MC_YUV,      // Luma, Crominancia U, Crominancia V
7      MC_YCrCb,    // Variante común usada en JPEG/MPEG
8      MC_CMYK,     // Cian, Magenta, Amarillo, Negro
9  };
10  ...
11  void convertir_a_modelo (int nfoto, int nres, modelo_color mc);
12  // Convierte la imagen a un espacio de color específico. Genera 3 o 4
13  // nuevas ventanas independientes, una para cada canal del modelo elegido

```

Listing 5: Declaracion de enum `modelo_color`.

Finalmente, realizamos la implementación de la función `convertir_a_modelo()`. Para realizar la conversión, basta con un `switch` del parámetro de la función `mc` y usar la operación `cvtColor()` de OpenCV. Para consultar el código completo, se remite al anexo de la implementación del método [A.2](#).

```

1  // imagenes.cpp
2  void convertir_a_modelo(int nfoto, int nres, modelo_color mc) {
3      ...
4      switch (mc) {
5          case MC_RGB:
6              cvtColor(img, res, COLOR_BGR2RGB);
7              break;
8
9          case MC_HSV:
10             cvtColor(img, res, COLOR_BGR2HSV);
11             break;
12             ...
13             case MC_CMYK:
14                 ...
15                 break;
16
17             default:
18                 res = img.clone();
19                 break;
20         }
21
22         vector<Mat> canales;
23         split(res, canales);
24
25         for (size_t i = 0; i < canales.size(); ++i)
26             crear_nueva(nres+i, canales[i])
27     }
28

```

Listing 6: Declaracion de enum `modelo_color`.

5.7. Perfilado

5.7.1. Introducción

El perfilado o *sharpening* es una técnica de procesamiento de imágenes utilizada para mejorar la acutancia (percepción de nitidez) mediante el aumento del contraste local en los bordes. [6]

Existen varias aproximaciones para implementar este efecto. Sin embargo, vamos a tomar de modelo la idea presentada en la teoría de la asignatura, que usa la segunda derivada de la imagen, utilizando el operador laplaciano. Los bordes en una imagen se caracterizan por cambios bruscos en la intensidad. Matemáticamente, estos cambios pueden detectarse mediante derivadas. Mientras que la primera derivada alcanza un máximo o mínimo en el borde, la segunda derivada cruza por cero (*zero-crossing*) en el punto de transición del borde. [7]

Sea $f(x,y)$ la función de intensidad de la imagen original, el operador Laplaciano, denotado como ∇^2 , es un operador diferencial isotrópico (invariante a la rotación) definido como la suma de las segundas derivadas parciales: [8]

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

En el dominio discreto, este operador se aproxima mediante la convolución de la imagen I con un núcleo (kernel) K . En nuestra aplicación, el usuario ajustará el kernel según el valor de `radio`. Un kernel común de 3×3 para la Laplaciana es:

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

La imagen final perfilada $g(x,y)$ se obtiene restando una fracción de la imagen Laplaciana a la imagen original. Esto tiene el efecto de resaltar las transiciones de intensidad:

$$g(x,y) = f(x,y) - c \cdot \nabla^2 f(x,y)$$

Donde:

- $\nabla^2 f(x,y)$ es el resultado de aplicar la función `cv::Laplacian`.
- c es un factor de ponderación controlado por el usuario (porcentaje).

5.7.2. Implementación

La implementación en C++ se ha llevado a cabo tomando de ejemplo la porción de código de las diapositivas de teoría. Como para cualquier nueva función, se define su prototipo en `imagenes.h`:

```
1 void perfilado(int nfoto, int radio, double porcentaje, bool guardar=false);
2 // Mostrar perfilado de una imagen a partir de un radio y un porcentaje
```

Listing 7: Declaración de enum `modelo_color`.

1. **Cálculo de la laplaciana con signo:** Como se ha mencionado, el operador laplaciano busca cambios bruscos entre los valores de los píxeles (bordes). Sin embargo, en QFotoPaint6 trabajamos por defecto con el formato `CV_8U` de OpenCV, es decir, enteros sin signo o *unsigned*. No podemos calcular la laplaciana sobre este formato ya que los valores negativos se truncarían a 0, por lo que realizamos un cambio de tipo para trabajar sobre una profundidad de 16 bits con signo:

```
1 Laplacian(img, lap, CV_16S, ksize);
2 img.convertTo(img16, CV_16S);
```

Listing 8: Declaracion de enum `modelo_color`.

2. **Operación aritmética:** Para realizar la resta $Img - c \cdot Lap$, la imagen original también debe convertirse temporalmente a 16 bits (`CV_16S`) para evitar desbordamientos, aunque al final se vuelve al formato de origen `CV_8U`.

```
1 img.convertTo(img16, CV_16S);
2 double factor = porcentaje / 100.0;
3 suma = img16 - (factor * lap);
4 suma.convertTo(res, CV_8U);
```

Listing 9: Declaracion de enum `modelo_color`.

Para ver el código completo, se remite al anexo de la implementación de la función [A.3](#).

5.8. Ajuste de los canales rojo, verde y azul

Esta funcionalidad permite la manipulación individual de los componentes cromáticos primarios de la imagen (rojo, verde y azul). Mediante operaciones lineales puntuales, el usuario puede corregir la imagen y aplicar efectos artísticos modificando la ganancia y el desplazamiento de cada canal.

5.8.1. Introducción

La operación se basa en la transformación lineal básica de intensidad. Para cada píxel (x, y) en un canal específico c (donde $c \in \{R, G, B\}$), la nueva intensidad $g_c(x, y)$ se calcula a partir de la intensidad original $f_c(x, y)$ mediante la ecuación lineal: [9]

$$g_c(x, y) = \alpha_c \cdot f_c(x, y) + \beta_c \quad (1)$$

Donde los parámetros controlados por el usuario actúan de la siguiente manera:

- **Ganancia (α_c):** Corresponde al contraste del canal. Un valor $\alpha > 1$ expande el rango dinámico (aclara los tonos claros, oscurece los oscuros), mientras que $0 < \alpha < 1$ lo comprime. Matemáticamente es una operación multiplicativa.
- **Desplazamiento (β_c):** Corresponde al brillo del canal. Un valor positivo suma luminosidad uniforme a todo el canal, mientras que un valor negativo la sustrae.

El objetivo de esta funcionalidad es realizar la operación de forma independiente para los tres canales.

5.8.2. Implementación

Definimos el prototipo de nuestra función en `imagenes.h` con los parámetros necesarios para realizar el cálculo:

```
1 void ver_rojoverdeazul(int nfoto,
2     double r_mul, double r_add,
3     double g_mul, double g_add,
4     double b_mul, double b_add,
5     bool guardar=false);
6 // Modifica (multiplicar/sumar) cada uno de los canales de forma
7 // independiente, para poder hacer una transformación de color
```

Listing 10: Declaración de enum `modelo_color`.

La implementación en C++ en el fichero `imagenes.cpp` tendrá el siguiente flujo de trabajo:

1. **Separación de Canales:** Dado que las imágenes se almacenan en formato entrelazado BGR en OpenCV, primero se separan los planos de color utilizando la función `cv::split`.

```
1 Mat canales[3];
2 split(foto[nfoto].img, canales);
```

2. **Transformación Lineal:** Se aplica la transformación utilizando el método nativo `cv::Mat::convertTo`. Esta operación, que ya ha sido usada anteriormente, permite ajustar los valores `alpha` y `beta` por parámetros. Internamente, ocurre lo siguiente: [10]

- Realiza la operación $pixel \cdot \alpha + \beta$ de forma vectorizada.
- Gestiona automáticamente la saturación si el resultado matemático supera 255 o es menor a 0, manteniendo el tipo de dato `CV_8U`.

```
1 canales[0].convertTo(canales[0], -1, b_mul, b_add);
2 canales[1].convertTo(canales[1], -1, g_mul, g_add);
3 canales[2].convertTo(canales[2], -1, r_mul, r_add);
```

3. **Fusión:** Finalmente, los canales procesados se recombinan en una única imagen utilizando `cv::merge`.

```
1 Mat res;
2 merge(canales, 3, res);
```

La interfaz de usuario (clase `Perfilado`) convierte los valores de los *sliders* (enteros) a los coeficientes flotantes necesarios, dividiendo por un factor de escala (100.0) para el parámetro α , permitiendo así precisión decimal en la multiplicación.

Para ver el código en su totalidad, se remite al anexo de la implementación de la clase [A.4](#).

6. Mejoras adicionales no mencionadas en el enunciado

6.1. Incremento del paso en arcoíris

6.1.1. Introducción

Como propuesta del profesor, se ha diseñado una solución para que el usuario sea capaz de modificar el “*paso*” con el que se cambia de color del pincel en la herramienta “Arcoíris”. Esta característica permitirá al usuario un mayor control sobre apariencia del efecto.

El “paso” se refiere a la velocidad de avance que en la secuencia de colores del espectro cada vez que se dibuja un píxel. Un paso pequeño provoca que los colores cambien lentamente, generando transiciones suaves y graduales entre tonos consecutivos. Por el contrario, un paso mayor acelera el cambio de color, produciendo un efecto más vivo.

6.1.2. Implementación

Para implementar esta funcionalidad, se ha definido un nuevo elemento `QWidget` en `mainwindow.ui`. Este elemento estará por defecto invisible, y su visibilidad cambiará al hacer *click* sobre el botón de la herramienta.

También se ha valorado la necesidad de ocultar el panel si se pierde el foco en él. Se ha tenido que recurrir a un *listener* de eventos de `QMouseEvent` ya que los paneles no proporcionan métodos `isHovered()`. Esta decisión es muy poco eficiente, pero aún así se ha decidido implementar.

```
1 // mainwindow.cpp
2 void MainWindow::on_toolButton_9_clicked() {
3     herr_actual = HER_ARCOIRIS;
4     ui->incrementoArcoirisWidget->setVisible(!ui->incrementoArcoirisWidget->
5     isVisible());
6 }
7 ...
8 void MainWindow::mouseMoveEvent(QMouseEvent *event) {
9     if (ui->incrementoArcoirisWidget->isVisible()) {
10         QPoint pos = event->pos();
11
12         // Si el ratón no está dentro del widget y tampoco sobre el botón
13         if (!ui->incrementoArcoirisWidget->geometry().contains(pos) &&
14             !ui->toolButton_9->geometry().contains(pos)) {
15             ui->incrementoArcoirisWidget->hide();
16             ui->toolButton_9->setDown(false);
17         }
18     }
19 }
```

Listing 11: Mostrar/Ocultar panel al presionar la herramienta

En la clase `imagenes` también se han realizado algunas modificaciones. La primera es la necesidad de tener una variable externa que se pueda utilizar en `mainwindow`, sustituyendo a la variable local `incremento`.

```
// imagenes.h
constexpr int DEFAULT_INCREMENTO_ARCOIRIS = 10;
extern int incremento_arcoiris;
```

```
// imagenes.cpp  
int incremento_arcoiris = DEFAULT_INCREMENTO_ARCOIRIS;
```

6.2. Rehacer última acción

6.2.1. Introducción

Con el fin de completar la funcionalidad de deshacer las acciones realizadas, y con la base ya implementada para soportarla, se ha optado por ofrecer al usuario una forma de recuperar los cambios “deshechos”. Esta operación es conocida con el nombre de rehacer o *redo* en inglés. La idea es la misma que para la operación de “deshacer”: modificar la estructura `ventana` con un nuevo tipo de dato que actúe exactamente igual que la pila implementada anteriormente en la figura 1. La forma es la siguiente:

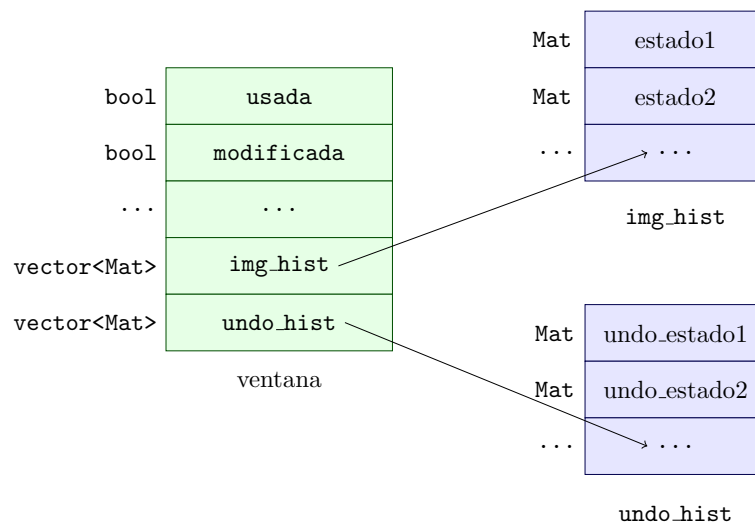


Figura 3: Actualización de la estructura `ventana` con un vector `undo_hist`.

6.2.2. Implementación

A nivel de código:

```

1  // imagenes.h
2  #define MAX_HIST_IMG 20    // Tamaño del historial para deshacer/rehacer
3  ...
4  struct ventana {
5      bool usada;           // Indica si se está usando actualmente
6      bool modificada;      // Ha sido modificada después de abrirla o crearla
7      string nombre;        // Nombre de la ventana codificada en UTF8
8      string nombref;       // Nombre del fichero de imagen codificado en Latin1
9      int orden;            // Número de orden entre las ventanas abiertas
10     Rect roi;              // Región de interés seleccionada
11     Mat img;               // Imagen almacenada en esta posición
12     vector<Mat> img_hist;  // Historial de imágenes
13     vector<Mat> undo_hist; // Historial de deshacer
14 };

```

Listing 12: Modificación del `struct ventana`.

Debemos actualizar la operación `deshacer` para que, en cada cambio de estado, se guarde la imagen actual. De esta manera, la imagen no se pierde y se guarda en la nueva pila. También será necesario definir una nueva operación `rehacer()`.


```

1 // imagenes.cpp
2 void deshacer (int nfoto) {
3     ventana& v = foto[nfoto];
4     ...
5     // guardar estado para poder ir hacia delante
6     v.undo_hist.push_back(v.img.clone());
7     ...
8     if (v.undo_hist.size() > MAX_HIST_IMG)
9         v.undo_hist.erase(v.undo_hist.begin());
10    ...
11 }
12
13 void rehacer (int nfoto) {
14     ventana& v = foto[nfoto];
15
16     if (v.undo_hist.empty()) {
17         qDebug() << "No hay más estados para rehacer.";
18         return;
19     }
20
21     // guardar estado en la pila de deshacer
22     v.img_hist.push_back(v.img.clone());
23
24     // recuperar y eliminar último estado de la pila de rehacer
25     v.img = v.undo_hist.back().clone();
26     v.undo_hist.pop_back();
27
28     imshow(v.nombre, v.img);
29 }

```

Listing 13: Modificación del `struct ventana`.

El atajo de teclado designado para el QAction correspondiente es `CTRL+Y`, el cual, a diferencia del atajo `CTRL+Z`, no se superpone con ningún otro y se puede ejecutar con el foco en imágenes de OpenCV.

7. Bitácora

Referencias

- [1] The Qt Company. Qclipboard class. <https://doc.qt.io/qt-6/qclipboard.html>, 2025. Accessed: 2025-11-29.
- [2] OpenCV Documentation. Histogram equalization — tutorial. https://docs.opencv.org/4.x/d4/d1b/tutorial_histogram_equalization.html, 2025. Accessed: 2025-11-29.
- [3] OpenCV Documentation. equalizeHist — image processing module. https://docs.opencv.org/4.x/d6/dc7/group_imgproc_hist.html#ga7e54091f0c937d49bf84152a16f76d6e, 2025. Accessed: 2025-11-29.
- [4] Wikipedia contributors. Histogram equalization — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Histogram_equalization#On_color_images, 2025. Accessed: 2025-11-29.
- [5] Wikipedia. Modelo de color cmyk — wikipedia, la enciclopedia libre. https://es.wikipedia.org/w/index.php?title=Modelo_de_color_CMYK&oldid=164414514, 2024. [Internet; descargado 31-diciembre-2024].
- [6] Wikipedia contributors. Unsharp masking — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Unsharp_masking&oldid=1317014229, 2025. [Online; accessed 2-December-2025].
- [7] Wikipedia contributors. Discrete laplace operator — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Discrete_Laplace_operator#Image_processing, 2025. [Online; accessed 2-December-2025].
- [8] OpenCV Documentation.
- [9] OpenCV. *Basic Linear Transformations — OpenCV 4.x tutorial*, 2025. Accessed: 2025-12-02.
- [10] OpenCV development team. *cv::Mat Class Reference — OpenCV 4.x*. OpenCV, 2025. Accessed: 2025-12-02.

A. Anexo

A.1. Código de la implementación de Información Multimedia

```
1 #include "informacionmultimedia.h"
2 #include "ui_informacionmultimedia.h"
3
4 #include "imagenes.h"
5
6 InformacionMultimedia::InformacionMultimedia(int numfoto, QWidget *parent)
7     : QDialog(parent)
8     , ui(new Ui::InformacionMultimedia)
9 {
10     ui->setupUi(this);
11     nfoto= numfoto;
12
13     mostrarInfo();
14 }
15
16 InformacionMultimedia::~InformacionMultimedia()
17 {
18     delete ui;
19 }
20
21 void InformacionMultimedia::mostrarInfo() {
22     ui->treeWidget->clear();
23
24     for (size_t i = 0; i < MAX_VENTANAS; ++i) {
25         Mat img = foto[i].img;
26
27         if (img.empty())
28             continue;
29
30         // elemento raíz
31         QTreeWidgetItem *root = new QTreeWidgetItem(ui->treeWidget);
32         QString nombreNodo = QString("%1")
33             .arg(foto[i].nombre);
34
35         QFont font = root->font(0);
36         font.setBold(true);
37         root->setFont(0, font);
38         root->setText(0, nombreNodo);
39
40         // dimensiones (ancho x alto)
41         QString dimensiones = QString("%1 x %2 píxeles")
42             .arg(img.cols)
43             .arg(img.rows);
44         QTreeWidgetItem *itemDimensiones = new QTreeWidgetItem(root);
45         itemDimensiones->setText(0, "Resolución (ancho x alto): " + dimensiones);
46
47         // total de píxeles
48         QString totalPíxeles = QString("Total de píxeles: %1").arg(img.total());
49         QTreeWidgetItem *itemTotal = new QTreeWidgetItem(root);
50         itemTotal->setText(0, totalPíxeles);
51
52         // relación de aspecto
```

```

51     int w = img.cols;
52     int h = img.rows;
53     double ratio = (double) w / h;
54     double delta = 0.01;
55     QString textoRatio;
56
57     if (abs(ratio - (16.0/9.0)) < delta) {
58         textoRatio = "16:9 (Panorámico)";
59     } else if (abs(ratio - (4.0/3.0)) < delta) {
60         textoRatio = "4:3 (Estándar TV)";
61     } else if (abs(ratio - 1.0) < delta) {
62         textoRatio = "1:1 (Cuadrado)";
63     } else if (abs(ratio - (3.0/2.0)) < delta) {
64         textoRatio = "3:2 (Fotografía Clásica/DSLR)";
65     } else if (abs(ratio - (21.0/9.0)) < delta) {
66         textoRatio = "21:9 (Ultra Panorámico)";
67     } else if (abs(ratio - (16.0/10.0)) < delta) {
68         textoRatio = "16:10";
69     } else {
70         textoRatio = QString("%1").arg(ratio, 0, 'f', 2);
71     }
72
73     QTreeWidgetItem *itemAspecto = new QTreeWidgetItem(root);
74     itemAspecto->setText(0, "Relación de Aspecto: " + textoRatio);
75
76     // tamaño en memoria
77     double bytes = (double)img.total() * img.elemSize();
78     QString memoria;
79
80     if (bytes >= pow(1024, 2))
81         memoria = QString::number(bytes / pow(1024, 2), 'f', 2) + " MB";
82     else memoria = QString::number(bytes / 1024.0, 'f', 2) + " KB";
83
84     QTreeWidgetItem *itemMemoria = new QTreeWidgetItem(root);
85     itemMemoria->setText(0, "Tamaño (en memoria): " + memoria);
86
87     // canales de color
88     QTreeWidgetItem *itemCanales = new QTreeWidgetItem(root);
89     itemCanales->setText(0, QString("Canales de color: %1").arg(img.channels
90 ()));
91
92     // tipo CV
93     QString tipoMatriz = QString::fromStdString(cvTypeToString(img.type()));
94     QTreeWidgetItem *itemTipo = new QTreeWidgetItem(root);
95     itemTipo->setText(0, "Tipo de matriz (profundidad/canales): " +
96 tipoMatriz);
97
98     // tamaño de elemento
99     QTreeWidgetItem *itemElemSize = new QTreeWidgetItem(root);
100     itemElemSize->setText(0, QString("Bytes por Píxel: %1").arg(img.elemSize
101 ()));
102 }
103
104 ui->treeWidget->expandAll();

```

```

104 string InformacionMultimedia::cvTypeToString(int type) {
105     string r;
106
107     uchar depth = type & CV_MAT_DEPTH_MASK;
108     uchar chans = 1 + (type >> CV_CN_SHIFT);
109
110     switch ( depth ) {
111     case CV_8U:  r = "8U"; break;
112     case CV_8S:  r = "8S"; break;
113     case CV_16U: r = "16U"; break;
114     case CV_16S: r = "16S"; break;
115     case CV_32S: r = "32S"; break;
116     case CV_32F: r = "32F"; break;
117     case CV_64F: r = "64F"; break;
118     default:     r = "UNKNOWN"; break;
119     }
120     r += "C" + to_string(chans);
121     return r;
122 }

```

Listing 14: Código de `informacionmultimedia.cpp`

A.2. Código de la implementación de `convertir_a_modelo()`

```

1 // imagenes.cpp
2 void convertir_a_modelo(int nfoto, int nres, modelo_color mc)
3 {
4     Mat img = foto[nfoto].img;
5     Mat res;
6
7     switch (mc) {
8     case MC_RGB:
9         cvtColor(img, res, COLOR_BGR2RGB);
10        break;
11
12    case MC_HSV:
13        cvtColor(img, res, COLOR_BGR2HSV);
14        break;
15
16    case MC_HLS:
17        cvtColor(img, res, COLOR_BGR2HLS);
18        break;
19
20    case MC_XYZ:
21        cvtColor(img, res, COLOR_BGR2XYZ);
22        break;
23
24    case MC_YUV:
25        cvtColor(img, res, COLOR_BGR2YUV);
26        break;
27
28    case MC_YCrCb:
29        cvtColor(img, res, COLOR_BGR2YCrCb);
30        break;
31

```

```

32     case MC_CMYK:
33     {
34         Mat f;
35         img.convertTo(f, CV_32F, 1.0/255.0);
36
37         // paso intermedio BGR -> CMYK
38         Mat cmy = Scalar(1.0, 1.0, 1.0) - f;
39
40         vector<Mat> p;
41         split(cmy, p);
42
43         // K = min(C', M', Y')
44         Mat K;
45         min(p[2], p[1], K);
46         min(K, p[0], K);
47
48         // CMY -> CMYK
49         Mat k_diff = 1.0 - K + 1e-4;
50
51         vector<Mat> cmyk = {
52             (p[2] - K) / k_diff * 255.0, // C = (C' - K) / (1-K)
53             (p[1] - K) / k_diff * 255.0, // M = (M' - K) / (1-K)
54             (p[0] - K) / k_diff * 255.0, // Y = (Y' - K) / (1-K)
55             K * 255.0
56         };
57
58         merge(cmyk, res);
59         res.convertTo(res, CV_8U);
60         break;
61     }
62
63
64     default:
65         res = img.clone();
66         break;
67     }
68
69     // separar canales
70     vector<Mat> canales;
71     split(res, canales);
72
73     // crear las nuevas imágenes
74     for (size_t i = 0; i < canales.size(); ++i) {
75         crear_nueva(nres+i, canales[i]);
76     }
77 }
78

```

Listing 15: Modificación del `struct ventana`.

A.3. Código de la implementación de perfilado()

```

1 // imagenes.cpp
2 void perfilado(int nfoto, int radio, double porcentaje, bool

```

```

guardar)
3      {
4          Mat img = foto[nfoto].img;
5          Mat res;
6
7          Mat lap, img16, suma;
8
9          // tamaño del kernel
10         int ksize = 2 * radio + 1;
11
12         // calcular laplaciana
13         Laplacian(img, lap, CV_16S, ksize);
14
15         img.convertTo(img16, CV_16S);
16
17         double factor = porcentaje / 100.0;
18
19         suma = img16 - (factor * lap);
20
21         suma.convertTo(res, CV_8U);
22
23         imshow(foto[nfoto].nombre, res);
24
25         if (guardar) {
26             guardar_estado(nfoto);
27             res.copyTo(img);
28             foto[nfoto].modificada = true;
29         }
30     }
31

```

Listing 16: Modificación del `struct ventana`.

A.4. Código de la implementación de ver_rojooverdeazul()

```

1          // imagenes.cpp
2          void ver_rojooverdeazul(int nfoto, double r_mul,
3          double r_add,
4          double g_mul, double g_add,
5          double b_mul, double b_add,
6          bool guardar)
7          {
8              Mat canales[3];
9              split(foto[nfoto].img, canales);
10
11             canales[0].convertTo(canales[0], -1, b_mul,
12             b_add);
13
14             canales[1].convertTo(canales[1], -1, g_mul,
15             g_add);
16
17             canales[2].convertTo(canales[2], -1, r_mul,
18             r_add);
19
20             Mat res;
21             merge(canales, 3, res);

```

```
16         imshow(foto[nfoto].nombre, res);
17
18
19         if (guardar) {
20             guardar_estado(nfoto);
21             res.copyTo(foto[nfoto].img);
22             foto[nfoto].modificada = true;
23         }
24     }
25
```

Listing 17: Modificación del `struct ventana`.